Video title: Creating Simple Datasets in PyTorch (Module 2)

Code snippets reference

This document lists the code snippets shown in the video along with brief descriptions. We recommend reviewing this document alongside the video for the best learning experience.

Disclaimer:
The code snippets shown in this video are for conceptual illustration only. They are simplified or partial examples designed to explain the underlying ideas and may not run as-is. Please avoid copying the code directly without understanding the context and structure required to make it work.

S. No.	Code	Description
1.	`from torch.utils.data import Dataset`	• **Slide title:** Importing and subclassing the dataset class • **Purpose**: Import the Dataset class and create a custom dataset by subclassing it.
2.	`class ToyDataset(Dataset):` `  pass`	• **Slide title:** Importing and subclassing the dataset class • **Purpose:** Define an empty custom dataset class that inherits functionality from the Dataset base class.

3.	`import torch`
	`class ToyDataset(Dataset):`
	`  def __init__(self):`
	`  self.x = torch.arange(100)`
	`  self.y = 2 * self.x`
	`  self.length = len(self.x)`
		• **Slide title:** Defining the dataset constructor
		• **Purpose:** Initialize dataset attributes and create sample feature and target tensors.

4.	`def __len__(self):` `  return self.length`	• **Slide title:** Implementing the len method • **Purpose:** Define how the dataset reports the total number of samples it contains.

5.	dataset = ToyDataset() print(len(dataset))	• **Slide title:** Implementing the len method • **Purpose:** Create a dataset object and check the number of samples using the len function.

6.	def __getitem__(self, index):
	  x = self.x[index]
	  y = self.y[index]
	  return (x, y)
		• **Slide title:** Accessing samples with the getitem method
		• **Purpose:** Implement a method that retrieves a specific data sample using an index.

7.	sample = dataset[0] print(sample)	• **Slide title:** Accessing samples with the getitem method • **Purpose:** Access and print a single dataset sample using indexing.
8.	for i in range(3): print(dataset[i])	• **Slide title:** Accessing samples with the getitem method • **Purpose:** Iterate through the dataset and retrieve multiple samples using a loop.

9.	class AddMultiply(object):
	  def __init__(self, add_x, mul_y):
	  self.add_x = add_x
	  self.mul_y = mul_y
		• **Slide title:** Creating transform classes
		• **Purpose:** Define a custom transform class that will modify dataset samples.

10.	def __call__(self, sample):
	  x, y = sample
	  x = x + self.add_x
	  y = y * self.mul_y
	  return (x, y)
		• **Slide title:** Creating transform classes
		• **Purpose:** Implement the callable method that applies transformations to each sample.

11.	transform = AddMultiply(1, 2) sample = transform(dataset[0])	• **Slide title:** Applying transforms to the dataset

		• **Purpose:** Apply the custom transform to a dataset sample to modify its values.

12.	class ToyDataset(Dataset):
	  def __init__(self, transform=None):
	  self.x = torch.arange(100)
	  self.y = 2 * self.x
	  self.length = len(self.x)
	  self.transform = transform
		• **Slide title:** Applying transforms to the dataset
		• **Purpose:** Modify the dataset constructor to accept an optional transform parameter.

13.	def __getitem__(self, index):
	  sample = (self.x[index], self.y[index])
	  if self.transform:
	  sample = self.transform(sample)
	  return sample
		• **Slide title:** Applying transforms to the dataset
		• **Purpose:** Apply the transform to samples when they are retrieved from the dataset.

14.	from torchvision import transforms composed = transforms.Compose([AddMultiply(1, 2)]) dataset = ToyDataset(transform=composed)	• **Slide title:** Composing multiple transforms • **Purpose:** Combine multiple transforms into a single pipeline using Compose.